Design and Analysis of a Direct-Mapped Cache Controller in Verilog

Asad Ali, Divyansh Bhartia, Vikrant Yadav, Mayank Pillai, Prashant Raj
Project 6 — Modern Computer Architecture

Abstract—A cache is a small, fast memory that keeps recently used data close to the processor so that most memory accesses do not have to wait for the slow main memory. We design, simulate and synthesise a 1 KB direct-mapped cache with 16-byte blocks and 32-bit addresses in Verilog. The design is split into the modules suggested for the project: an address decoder, valid-bit, tag and data RAMs, a tag comparator, a four-state controller finite-state machine (FSM) and a main-memory interface. Reads use a refill-on-miss policy and writes use write-through with no-write-allocate. A self-checking testbench replays memory-access traces (sequential, repeated, random, conflict and mixed read/write), compares every value read with a reference memory and reports hit ratio, miss ratio and average memory access time (AMAT). With a 10-cycle main memory the hit time is 2 cycles and the miss penalty 12 cycles. The sequential trace reaches a 75% hit ratio (AMAT 5.0 cycles), the repeated loop 97.5% (2.3 cycles), random accesses 1.1% (13.9 cycles) and the conflict trace 0% (14.0 cycles). The AMAT measured in simulation equals the formula value in every case and no data errors were found. Yosys synthesis for a Xilinx 7-series FPGA uses 316 LUTs, 138 flip-flops and 136 distributed-RAM primitives.

Index Terms—cache memory, direct-mapped cache, cache controller, finite-state machine, AMAT, Verilog, locality

I. Introduction

Processors run much faster than main memory (DRAM). If every load and store waited for DRAM, the processor would spend most of its time idle. Caches solve this by keeping a copy of recently used data in a small, fast memory next to the processor [1]. Caches work because programs show locality [7]:

- Temporal locality — data used now is likely to be used again soon (e.g. loop variables).
- Spatial locality — data near a used address is likely to be used soon (e.g. array elements).

The simplest cache organisation is the direct-mapped cache, in which every memory block can be stored in exactly one cache line. This makes the lookup a single tag comparison, so the hardware is small and fast.

The goal of this project is to design a direct-mapped cache controller in Verilog and to analyse its hit/miss behaviour on representative memory-access traces. Our contributions are:

- a modular, synthesisable Verilog cache (1 KB, 16-byte blocks, 32-bit addresses) with a 4-state controller;
- a self-checking testbench and six memory-access traces that show the effect of locality and conflicts;
- hit ratio, miss ratio and AMAT measurements, a memory-latency sweep and FPGA synthesis results.

II. Literature Review

Cache fundamentals. Smith's survey [3] describes the main design choices of a cache: placement (direct-mapped, set-associative, fully associative), block size, replacement and write policy. Hennessy and Patterson [1] define the average memory access time,

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}, \qquad (1)$$

which we use as the main performance metric. They also classify misses into the "three Cs": compulsory (first access to a block), capacity (the working set does not fit) and conflict (two blocks compete for the same line).

Direct-mapped versus associative caches. Hill [4] argued that direct-mapped caches can outperform set-associative ones in practice, because their shorter hit time can outweigh their higher miss ratio. Hill and Smith [5] measured how associativity changes miss ratios and found that moving from direct-mapped to two-way removes a large part of the conflict misses. Jouppi [6] kept the fast direct-mapped cache and removed many conflict misses by adding a small fully associative victim cache.

Cache controller design. Patterson and Hennessy [2] present a simple cache controller as a four-state FSM (Idle, Compare Tag, Write-Back, Allocate) for a write-back cache. We follow the same structure, but replace the write-back state with a write-to-memory state because we use a write-through policy, which needs no dirty bits and is easier to verify.

Tools. The design is written in Verilog (IEEE 1364 [8]), simulated with Icarus Verilog [9] and synthesised with Yosys [10], which are free, open-source tools.

III. Proposed Solution

A. Cache parameters and address fields

The cache stores 1024 bytes in blocks of 16 bytes, so it has $1024/16 = 64$ lines. A 32-bit byte address is split into three fields (Table I):

- Offset (4 bits): $\log_2 16 = 4$; bits [3:2] choose one of the 4 words in a block.
- Index (6 bits): $\log_2 64 = 6$; chooses the cache line.
- Tag (22 bits): the remaining $32 - 6 - 4$ bits; identifies which memory block is in the line.

The total storage is $64 \times (128 + 22 + 1) = 9664$ bits, of which 8192 are data. Addresses that differ by a multiple of 1 KB have the same index and therefore conflict for the same line.

TABLE I
Address breakdown (32-bit address)

Field	Tag	Index	Offset
Bits	31–10	9–4	3–0
Width	22	6	4
Example 0x0000_0404	0x000001	0	4 (word 1)

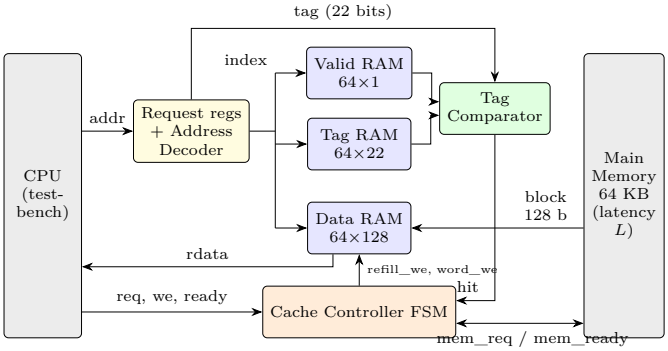


Fig. 1. Block diagram of the direct-mapped cache.

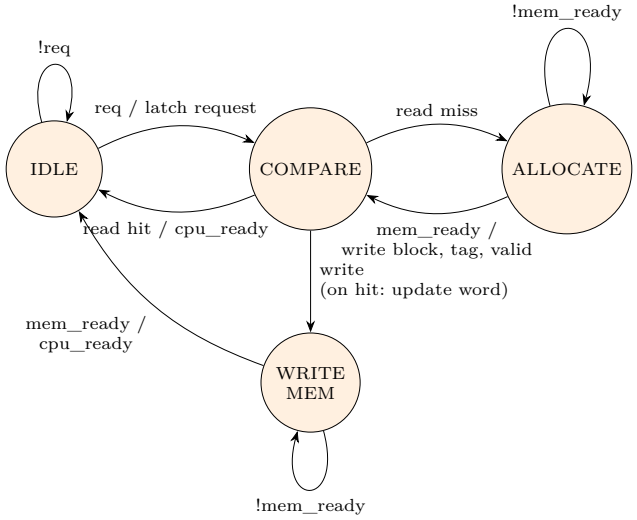


Fig. 2. Cache controller FSM.

B. Architecture

Fig. 1 shows the block diagram. Each box is one Verilog module:

- address_decoder — splits the latched address into tag, index and word select (wiring only).
- valid_ram — 64 valid bits, all cleared on reset, so the cache starts empty.
- tag_ram — 64 tags of 22 bits.
- data_ram — 64 blocks of 128 bits; can write a whole block (refill) or one word (store hit).
- tag_comparator — hit = valid ∧ (stored tag = address tag).
- cache_controller — the FSM that sequences lookup, refill and memory writes.
- main_memory — a 64 KB memory model with a configurable latency L (10 cycles by default), that returns a whole 16-byte block on a read.
- cache_top — connects the modules and holds the request registers.

All RAMs are read asynchronously, so the tag check and data read finish in one clock cycle.

C. Controller FSM

The controller (Fig. 2) has four states:

1) IDLE: wait for cpu_req; latch the address, write data and read/write flag.
2) COMPARE: check the valid bit and the tag. On a read hit the word is returned and cpu_ready is raised. On a read miss the FSM goes to ALLOCATE. On a write, the word is updated in the cache if it hits and the FSM goes to WRITE_MEM.
3) ALLOCATE: request the block from main memory. When mem_ready arrives, the block, tag and valid bit are written in one cycle and the FSM returns to COMPARE, which now hits.
4) WRITE_MEM: write the word to main memory (write-through), then raise cpu_ready.

Write misses do not load the block into the cache (no-write-allocate). The controller also outputs one-cycle stat_hit/stat_miss pulses on the first comparison of each request, which the testbench counts.

D. Timing

A hit takes 2 cycles (IDLE → COMPARE → IDLE). A read miss takes $2 + (L + 2) = 14$ cycles for $L = 10$: IDLE and COMPARE as for a hit, then $L + 1$ cycles in ALLOCATE (L cycles of memory latency plus the cycle in which the block is written), and a second COMPARE that now hits. Hence the hit time is 2 cycles and the miss penalty is $L + 2 = 12$ cycles, so by (1)

$$\text{AMAT} = 2 + \text{Miss rate} \times 12 \text{ cycles}. \tag{2}$$

E. Verification method and traces

The testbench (tb_cache.v) reads a trace file in which each line is R addr or W addr data. Main memory is initialised so that the word at address A holds the value A, which makes results easy to check by eye. The testbench keeps its own reference memory, checks every value read, counts hits, misses and cycles, and writes a per-access log and a CSV summary. A Python script generates six traces:

- sequential: read 4 KB word by word (spatial locality);
- repeated: read a 512-byte array 10 times (temporal locality);
- random: 1000 random words in 64 KB (little locality);
- conflict: alternate between 0x040 and 0x440, which share line 4;
- mixed: 200 reads and writes (30% writes) in 2 KB, to test write-through;

- demo: 8 hand-picked accesses that show a cold miss, hits, a conflict miss and a write hit.

IV. Results and Discussion

A. Functional verification

All six traces finish with zero data errors, so every read returned the correct value, including reads after writes. Fig. 3 shows the start of the demo trace: the first read of 0x000 misses (cold miss, valid bit 0), the FSM spends 10 cycles in ALLOCATE waiting for memory, and the next reads of 0x004 and 0x00C hit in 2 cycles because they are in the block that was just loaded. Later in the trace, 0x400 replaces block 0x000 in line 0 (same index, different tag), so the following read of 0x000 misses again.

B. Hit ratio and AMAT

Table II and Fig. 4 give the results for $L = 10$. The AMAT measured from the simulated cycle counts equals the value from (1) for every trace.

TABLE II
Results for a 10-cycle main memory (reads only)

Trace	Reads	Misses	Hit ratio	AMAT (cyc.)
sequential	1024	256	75.0%	5.00
repeated	1280	32	97.5%	2.30
random	1000	989	1.1%	13.87
conflict	200	200	0.0%	14.00
mixed	142	100	29.6%	10.45

- Sequential (75%): each block holds 4 words, so the first word misses and the next three hit. This is spatial locality: the hit ratio is $1 - 1/4$ and would rise with larger blocks.
- Repeated (97.5%): the 512-byte array (32 blocks) fits in the 1 KB cache. Only the 32 compulsory misses of the first pass occur, so the hit ratio is $1 - 32/1280$. This is temporal locality.
- Random (1.1%): the cache holds only 1/64 of the 64 KB address range, so almost every access is a capacity or compulsory miss.
- Conflict (0%): the two addresses map to the same line and evict each other on every access, although the rest of the cache is empty. This is the main weakness of direct mapping; a 2-way set-associative cache or a victim cache [6] would turn all but the first two accesses into hits.
- Mixed (29.6%): the 2 KB region is twice the cache size and write misses do not allocate, so many reads miss. Of the 58 writes, 20 hit and updated the cache; all 58 were written to memory.

C. Effect of the miss penalty

Fig. 5 repeats the read-only traces for memory latencies from 5 to 100 cycles. For the repeated trace AMAT grows only from 2.18 to 4.55 cycles, because only 2.5% of accesses pay the penalty. For the random and conflict traces AMAT grows almost one-for-one with the latency (from about 9 to 103–104 cycles). A high hit ratio is therefore more and more important as the gap between processor and memory speed grows.

D. Synthesis

The cache (without the main-memory model) was synthesised with Yosys for a Xilinx 7-series FPGA (Table III). The 64-entry RAMs are mapped to distributed (LUT) RAM because they are read asynchronously. The flip-flops hold the request registers, the 64 valid bits and the FSM state. Most of the 197 input and 103 output buffers come from the wide 128-bit memory bus, which would be internal in a full processor.

TABLE III
Yosys synthesis results (Xilinx 7-series)

Resource	Count
LUTs (LUT2–LUT6)	316
Flip-flops (FDRE)	138
Distributed RAM (RAM64M)	136
CARRY4	2
I/O buffers (IBUF/OBUF)	197 / 103

E. Limitations

The main memory is a behavioural model with a fixed latency, accesses are whole 32-bit words, and the CPU waits for each access to finish (no pipelining). Write-through sends every store to memory; a write-back cache with dirty bits would reduce memory traffic for programs with many stores.

V. Individual Contributions

Table IV lists the main work of each member. Each member committed their own modules to the shared GitHub repository, and all members reviewed the integrated design and the report.

TABLE IV
Individual contributions

Member	Contribution
Asad Ali	Tag/index/offset derivation; address_decoder and tag_comparator modules
Divyansh Bhartia	Cache storage: data_ram, tag_ram, valid_ram with reset of valid bits
Vikrant Yadav	main_memory model and memory interface; trace generator (gen_traces.py)
Mayank Pillai	Controller FSM (cache_controller) and top-level integration (cache_top)
Prashant Raj	Self-checking testbench, hit/miss statistics, AMAT and latency sweep, Yosys synthesis

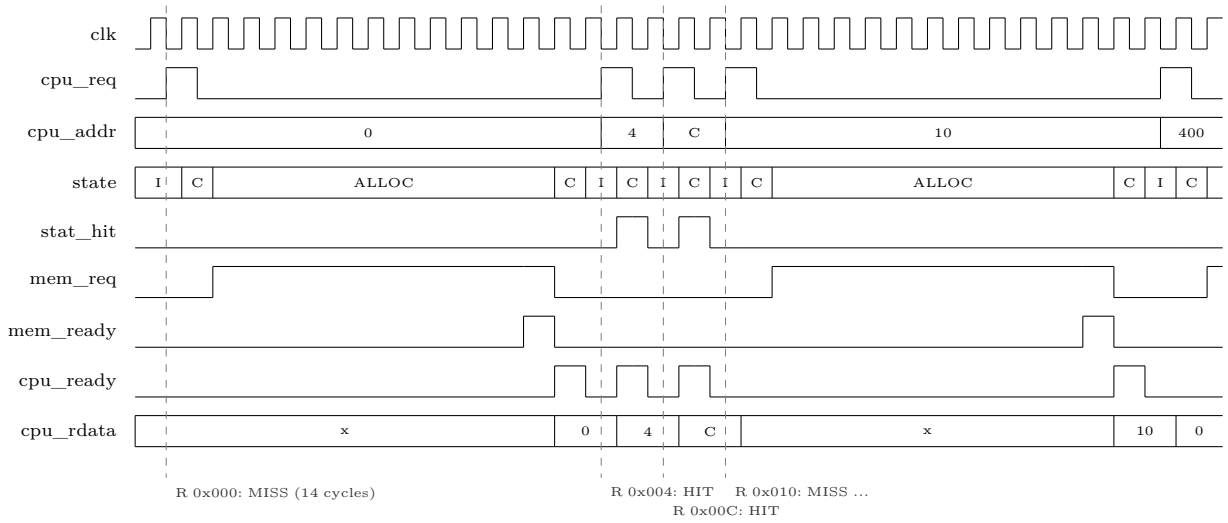


Fig. 3. Simulation waveform of the first accesses of the demo trace (I = IDLE, C = COMPARE, A = ALLOCATE).

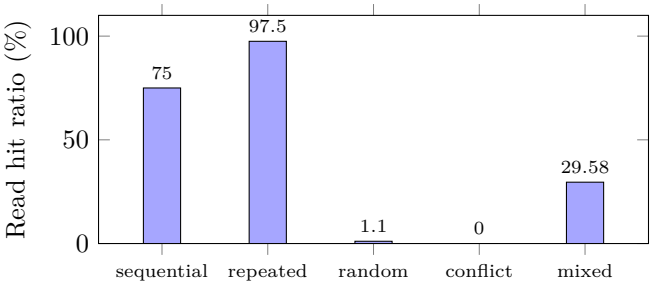


Fig. 4. Read hit ratio for each trace.

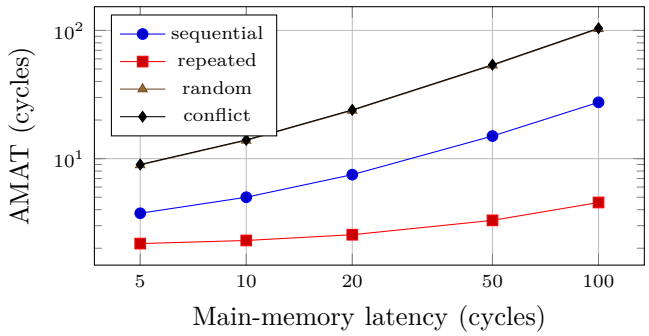


Fig. 5. Measured AMAT against main-memory latency (log scales). The random and conflict curves almost overlap.

VI. Conclusion

We designed a 1 KB direct-mapped cache with 16-byte blocks in Verilog, built from an address decoder, valid, tag and data RAMs, a tag comparator and a four-state controller FSM. A self-checking testbench confirmed correct lookup, refill, valid-bit handling and write-through on six traces with no data errors. The results match the theory exactly: sequential access gives a 75% hit ratio from spatial locality, a loop that fits in the cache gives 97.5% from temporal locality, and two addresses 1 KB apart give 0% because of conflict misses. The measured AMAT equals $2 + \text{miss rate} \times 12$ cycles for a 10-cycle memory, and the design fits in about 316 LUTs and 138 flip-flops on a Xilinx 7-series FPGA. Future work includes a 2-way set-associative version with LRU replacement to remove the conflict misses, a write-back policy, and a study of block and cache size.

References

[1] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[2] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.

[3] A. J. Smith, “Cache memories,” ACM Computing Surveys, vol. 14, no. 3, pp. 473–530, 1982.

[4] M. D. Hill, “A case for direct-mapped caches,” Computer, vol. 21, no. 12, pp. 25–40, Dec. 1988.

[5] M. D. Hill and A. J. Smith, “Evaluating associativity in CPU caches,” IEEE Trans. Computers, vol. 38, no. 12, pp. 1612–1630, Dec. 1989.

[6] N. P. Jouppi, “Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers,” in Proc. 17th Annu. Int. Symp. Computer Architecture (ISCA), 1990, pp. 364–373.

[7] P. J. Denning, “The locality principle,” Communications of the ACM, vol. 48, no. 7, pp. 19–24, 2005.

[8] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.

[9] S. Williams, “Icarus Verilog.” [Online]. Available: https://steveicarus.github.io/iverilog/

[10] C. Wolf, “Yosys Open SYnthesis Suite.” [Online]. Available: https://yosyshq.net/yosys/